

Harnessing Variational Auto-Encoders for Fitting Deep Mixed-Effects Models

Dimitris Rizopoulos, Nina van Gerwen, Sten Willemsen

Department of Biostatistics, Erasmus Medical Center Rotterdam



`d.rizopoulos@erasmusmc.nl`

1 Background & Motivation

High-dimensional Intensive Longitudinal Data

High-dimensional: many longitudinal outcomes

Intensive: many longitudinal measurements per sample unit

- Examples:
 - ▷ electronic health record data
 - ▷ intensive care monitor data

1 Background & Motivation (cont'd)

Growlt! App Data: Daily mood changes in young adults

- ▷ 1174 participants
 - * 123K planned measurements
 - * *75K measurements missing*

- ▷ Outcomes
 - * negative affect (bounded continuous $\in [1, 7]$)
 - * loneliness and boredom (dichotomous)

1 Background & Motivation (cont'd)

Objective: Dynamic predictions to guide future interventions

1 Background & Motivation (cont'd)

Common methods

▷ Multilevel/Random-Effects Models

↑ interpretable

↑ valid under MAR

↓ scale bad & parametric

▷ Machine Learning (e.g., recurrent neural networks)

↑ scale well

↑ semi-parametric

↓ valid under MCAR

1 Background & Motivation (cont'd)

Via a novel combination of machine learning and statistical ideas,

we wish to retain the advantageous features of random effects models, and

overcome the aforementioned difficulties

Deep Generalized Mixed Effects Models

2 Random Effects Models

Random Effects Models

- *Conditional Independence Assumptions:* Interrelationships
 - ▷ between longitudinal outcomes
 - ▷ within a longitudinal outcomeexplained via *latent variables*

2 Random Effects Models (cont'd)

General definition

$$\left\{ \begin{array}{l} [\mathbf{Y}_i \mid \mathbf{b}_i] \sim \mathcal{F}_\sigma, \\ g\{E(\mathbf{Y}_i \mid \mathbf{b}_i)\} = \mathbf{X}_i\boldsymbol{\beta} + \mathbf{Z}_i\mathbf{b}_i, \\ \mathbf{b}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{D}) \end{array} \right.$$

where

- ▷ $\mathbf{Y}_i$ response individual i
- ▷ $g(\cdot)$ link function
- ▷ $\mathcal{F}_\sigma$ conditional distribution

3 Deep Generalized Mixed Models

Definition

$$\left\{ \begin{array}{l} \mathbf{b}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \\ [\mathbf{y}_i \mid \mathbf{b}_i, \mathbf{X}_i, \mathbf{T}_i; \boldsymbol{\theta}] \sim \mathcal{F}(g\{\boldsymbol{\mu}_1(\mathbf{X}_i, \mathbf{T}_i; \boldsymbol{\beta}) + \boldsymbol{\mu}_2(\mathbf{B}_i, \mathbf{T}_i; \boldsymbol{\beta})\}, \sigma), \\ \boldsymbol{\mu}_1(\mathbf{X}_i, \mathbf{T}_i; \boldsymbol{\beta}) = \text{dnn}\{(\mathbf{X}_i, \mathbf{T}_i); \boldsymbol{\beta}_1, \boldsymbol{\lambda}_{\beta 1}\}, \\ \boldsymbol{\mu}_2(\mathbf{B}_i, \mathbf{T}_i; \boldsymbol{\beta}) = \text{dnn}\{(\mathbf{B}_i, \mathbf{T}_i); \boldsymbol{\beta}_2, \boldsymbol{\lambda}_{\beta 2}\}, \end{array} \right.$$

where

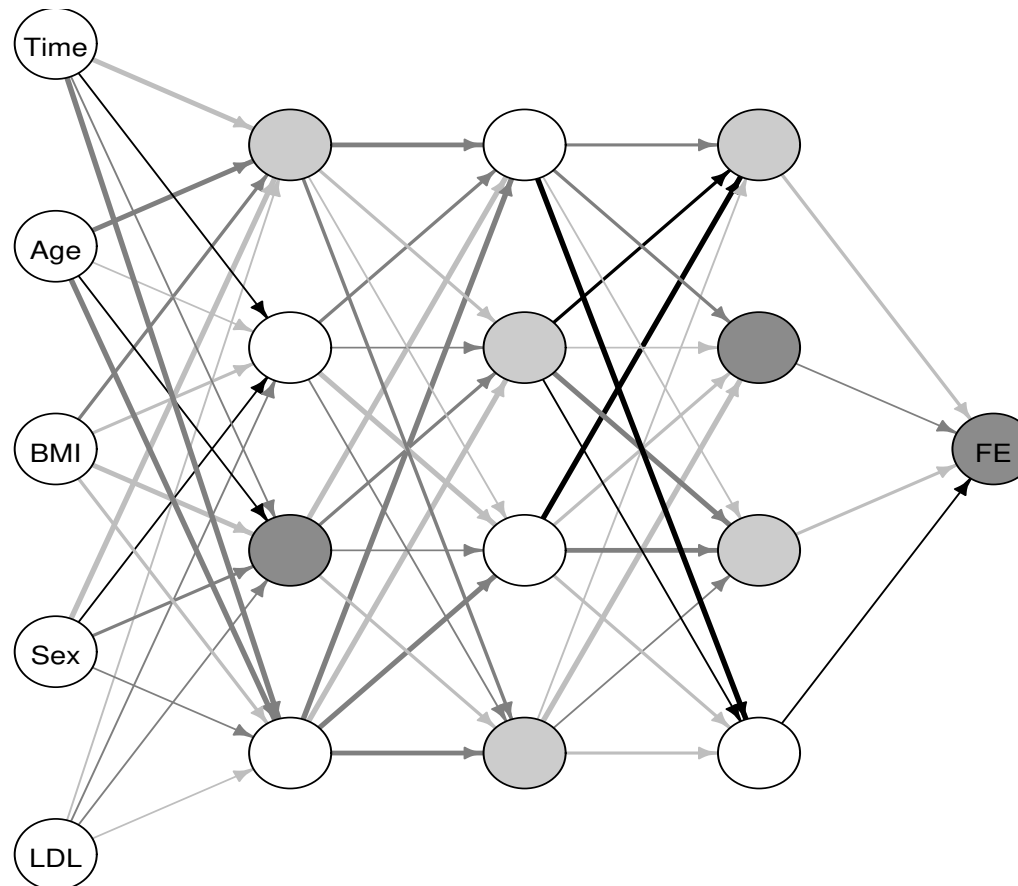
- ▷ $\text{dnn}(\mathbf{X}; \boldsymbol{\beta}, \boldsymbol{\lambda})$ denotes a dense deep neural network with
- ▷ input variables $\mathbf{x}$, parameters $\boldsymbol{\beta}$ and hyper-parameters $\boldsymbol{\lambda}$

3 Deep Generalized Mixed Models (cont'd)

- Notes:

- ▷ $\mu_1(\mathbf{X}_i, \mathbf{T}_i; \boldsymbol{\beta})$ *fixed effects neural network*
- ▷ $\mu_2(\mathbf{B}_i, \mathbf{T}_i; \boldsymbol{\beta})$ *random effects neural network*
- ▷ we redefine $\mathbf{X}_i$ excluding the time variable t_i
- ▷ matrix $\mathbf{T}_i$ contains functions of time

3 Deep Generalized Mixed Models (cont'd)



3 Deep Generalized Mixed Models (cont'd)

Dense Deep Neural Networks

$$\text{dnn}(x; \beta, \lambda) = f_1(\beta_1) \circ \sigma \circ f_2(\beta_2) \circ \dots \circ \sigma \circ f_L(\beta_L)$$

- ▷ layers $f_1, \dots, f_L$: tunable affine functions
- ▷ activation function σ : a simple fixed nonlinear function
- ▷ hyper-parameters λ : number of layers, nodes, ...

Semi-parametric nonlinear interaction models

4 DGMMs Estimation

- Estimation of the model parameters $\boldsymbol{\theta} = (\boldsymbol{\beta}^\top, \sigma)^\top$ requires the marginal likelihood function

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^n \log\{p(\mathbf{y}_i; \boldsymbol{\theta})\} = \sum_{i=1}^n \log \int p(\mathbf{y}_i | \mathbf{b}_i; \boldsymbol{\theta}) p(\mathbf{b}_i) d\mathbf{b}_i,$$

where

- ▷ $p(\mathbf{y}_i | \mathbf{b}_i; \boldsymbol{\theta})$ probability density or probability mass function of $\mathcal{F}(\boldsymbol{\beta}, \sigma)$
- ▷ $p(\mathbf{b}_i)$ probability density function of the $\mathcal{N}(\mathbf{0}, \mathbf{I})$

Computational challenge: $\log\{p(\mathbf{y}_i | \mathbf{b}_i; \boldsymbol{\theta})\}$ is nonlinear in the random effects $\mathbf{b}_i$

4 DGMMs Estimation (cont'd)

We propose an adaption of variational autoencoders to overcome this challenge

- We specify a tractable *variational distribution* for the random effects $q(\mathbf{b}_i)$
 - ▷ from which we can easily sample

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^n \log \int \frac{p(\mathbf{y}_i \mid \mathbf{b}_i; \boldsymbol{\theta}) p(\mathbf{b}_i)}{q(\mathbf{b}_i)} q(\mathbf{b}_i) d\mathbf{b}_i$$

4 DGMMs Estimation (cont'd)

- We use Jensen's inequality to push the logarithm inside the integral

$$\begin{aligned}\ell(\boldsymbol{\theta}) &= \sum_{i=1}^n \log \int \frac{p(\mathbf{y}_i \mid \mathbf{b}_i; \boldsymbol{\theta}) p(\mathbf{b}_i)}{q(\mathbf{b}_i)} q(\mathbf{b}_i) \, d\mathbf{b}_i \\ &\geq \sum_{i=1}^n \int \log \left\{ \frac{p(\mathbf{y}_i \mid \mathbf{b}_i; \boldsymbol{\theta}) p(\mathbf{b}_i)}{q(\mathbf{b}_i)} \right\} q(\mathbf{b}_i) \, d\mathbf{b}_i \\ &= \sum_{i=1}^n \left\{ \int \log \{p(\mathbf{y}_i \mid \mathbf{b}_i; \boldsymbol{\theta})\} q(\mathbf{b}_i) \, d\mathbf{b}_i - \int \log \left\{ \frac{q(\mathbf{b}_i)}{p(\mathbf{b}_i)} \right\} q(\mathbf{b}_i) \, d\mathbf{b}_i \right\} \\ &= \tilde{\ell}(\boldsymbol{\theta})\end{aligned}$$

4 DGMMs Estimation (cont'd)

- When we set $q(\mathbf{b}_i) = p(\mathbf{b}_i \mid \mathbf{y}_i; \boldsymbol{\theta})$, we obtain that

$$\tilde{\ell}(\boldsymbol{\theta}) = \ell(\boldsymbol{\theta})$$

Hence, we need a variational density $q(\mathbf{b}_i)$ that approximates $p(\mathbf{b}_i \mid \mathbf{y}_i; \boldsymbol{\theta})$

4 DGMMs Estimation (cont'd)

Bayesian central limit theorem: As $n_i \rightarrow \infty$

$$[\mathbf{b}_i \mid \mathbf{y}_i; \boldsymbol{\theta}] \sim \mathcal{N}\{\boldsymbol{\mu}_b(\mathbf{y}_i; \boldsymbol{\theta}), \boldsymbol{\Sigma}_b(\mathbf{y}_i; \boldsymbol{\theta})\}$$

with

$$\boldsymbol{\mu}_b(\mathbf{y}_i; \boldsymbol{\theta}) = \underset{\mathbf{b}}{\operatorname{argmax}} \left[\log\{p(\mathbf{y}_i \mid \mathbf{b}; \boldsymbol{\theta})\} + \log\{p(\mathbf{b})\} \right],$$

and

$$\boldsymbol{\Sigma}_b(\mathbf{y}_i; \boldsymbol{\theta}) = \left\{ -\frac{\partial^2}{\partial \mathbf{b}_i \partial \mathbf{b}_i^\top} \left[\log\{p(\mathbf{y}_i \mid \mathbf{b}_i; \boldsymbol{\theta})\} + \log\{p(\mathbf{b}_i)\} \right] \Big|_{\mathbf{b}_i = \boldsymbol{\mu}_b} \right\}^{-1}$$

4 DGMMs Estimation (cont'd)

- Both $\mu_b(\mathbf{y}_i; \boldsymbol{\theta})$ and $\Sigma_b(\mathbf{y}_i; \boldsymbol{\theta})$ can be seen as nonlinear functions of $\mathbf{y}_i$
- We approximate both parameters using deep neural networks again

$$\left\{ \begin{array}{l} [\mathbf{b}_i \mid \mathbf{y}_i; \boldsymbol{\phi}] \sim \mathcal{N}\{\boldsymbol{\mu}_b(\mathbf{y}_i; \boldsymbol{\phi}), \boldsymbol{\Sigma}_b(\mathbf{y}_i; \boldsymbol{\phi})\}, \\ \{\boldsymbol{\mu}_b(\mathbf{y}_i; \boldsymbol{\phi}), \boldsymbol{\Sigma}_b(\mathbf{y}_i; \boldsymbol{\phi})\} = \text{dnn}(\mathbf{y}_i; \boldsymbol{\phi}, \boldsymbol{\lambda}_{\boldsymbol{\phi}}) \end{array} \right.$$

Note: The parameter vector $\boldsymbol{\phi}$ is independent from $\boldsymbol{\theta}$

4 DGMMs Estimation (cont'd)

- $\text{dnn}(\mathbf{y}_i; \phi, \lambda_\phi)$ a deep neural network that estimates
 - ▷ a $(n \times k)$ matrix of means, and
 - ▷ a $(n \times k')$ matrix of variance components
- $\Sigma_b(\mathbf{y}_i; \phi)$ may be taken as a diagonal or a full covariance matrix
 - ▷ k' denotes the number of variance components

4 DGMMs Estimation (cont'd)

Glossary:

- ▷ $[b_i \mid y_i; \phi]$ is called the *encoder*
 - * because it encodes the observed data into a set of latent variables (*dimension reduction*)

- ▷ $[y_i \mid b_i, X_i, T_i; \theta]$ is called the *decoder*
 - * because it decodes the latent variables to predict the observed data

- ▷ *Amortized inference*
 - * we approximate $\mu_b(y_i; \theta)$ and $\Sigma_b(y_i; \theta)$ using the parameters ϕ that are independent of i

4 DGMMs Estimation (cont'd)

- *Loss function:* We minimize

$$-\tilde{\ell}(\boldsymbol{\theta}, \boldsymbol{\phi}) = -\sum_{i=1}^n \left\{ \int q(\mathbf{b}_i \mid \mathbf{y}_i; \boldsymbol{\phi}) \log\{p(\mathbf{y}_i \mid \mathbf{b}_i; \boldsymbol{\theta})\} d\mathbf{b}_i + \right. \\ \left. \int q(\mathbf{b}_i \mid \mathbf{y}_i; \boldsymbol{\phi}) \log\left\{\frac{q(\mathbf{b}_i \mid \mathbf{y}_i; \boldsymbol{\phi})}{p(\mathbf{b}_i)}\right\} d\mathbf{b}_i \right\} + \kappa(\boldsymbol{\theta}^\top \boldsymbol{\theta} + \boldsymbol{\phi}^\top \boldsymbol{\phi}),$$

with respect to $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$

4 DGMMs Estimation (cont'd)

- *Loss function*: First term using Monte Carlo

$$\begin{aligned}\int q(\mathbf{b}_i \mid \mathbf{y}_i; \phi) \log\{p(\mathbf{y}_i \mid \mathbf{b}_i; \boldsymbol{\theta})\} d\mathbf{b}_i &\approx \frac{1}{M} \sum_{m=1}^M \log\{p(\mathbf{y}_i \mid \mathbf{b}_i^{(m)}; \boldsymbol{\theta})\} \\ &= \frac{1}{M} \sum_{m=1}^M \sum_{j=1}^{n_i} \log\{p(y_{ij} \mid \mathbf{b}_i^{(m)}; \boldsymbol{\theta})\},\end{aligned}$$

with $(\mathbf{b}_i^{(1)}, \dots, \mathbf{b}_i^{(M)})$ denoting a sample from $q(\mathbf{b}_i \mid \mathbf{y}_i; \phi)$

4 DGMMs Estimation (cont'd)

- *Loss function*: Second term

$$\int q(\mathbf{b}_i \mid \mathbf{y}_i; \phi) \log \left\{ \frac{q(\mathbf{b}_i \mid \mathbf{y}_i; \phi)}{p(\mathbf{b}_i)} \right\} d\mathbf{b}_i$$

is the *Kullback-Leibler (KL) divergence* between the variational distribution $q(\mathbf{b}_i \mid \mathbf{y}_i; \phi)$ and the marginal distribution $p(\mathbf{b}_i)$

4 DGMMs Estimation (cont'd)

- *Loss function*: Second term in closed-form

▷ for a diagonal $\Sigma_b(\mathbf{y}_i; \phi)$ we obtain,

$$\int q(\mathbf{b}_i | \mathbf{y}_i; \phi) \log \left\{ \frac{q(\mathbf{b}_i | \mathbf{y}_i; \phi)}{p(\mathbf{b}_i)} \right\} d\mathbf{b}_i = \frac{1}{2} \sum_{l=1}^k \mu_{il}^2 + \sigma_{il}^2 - 1 - \log(\sigma_{il}^2)$$

▷ for a full covariance matrix

$$\int q(\mathbf{b}_i | \mathbf{y}_i; \phi) \log \left\{ \frac{q(\mathbf{b}_i | \mathbf{y}_i; \phi)}{p(\mathbf{b}_i)} \right\} d\mathbf{b}_i = \frac{1}{2} \left(\text{trace}\{\Sigma_b(\mathbf{y}_i; \phi)\} - k + \{\boldsymbol{\mu}_b(\mathbf{y}_i; \phi)\}^\top \{\boldsymbol{\mu}_b(\mathbf{y}_i; \phi)\} - \log[\det\{\Sigma_b(\mathbf{y}_i; \phi)\}] \right)$$

4 DGMMs Estimation (cont'd)

- *Loss function:* Last term

$$\kappa(\boldsymbol{\theta}^\top \boldsymbol{\theta} + \boldsymbol{\phi}^\top \boldsymbol{\phi})$$

- ▷ a ridge penalty to stabilize the parameter estimates and account for overfitting
- ▷ κ denoting the penalty parameter

4 DGMMs Estimation (cont'd)

Missing Data Challenge

▷ We approximate $q(\mathbf{b}_i \mid \mathbf{y}_i; \phi)$ as

$$\left\{ \begin{array}{l} [\mathbf{b}_i \mid \mathbf{y}_i; \phi] \sim \mathcal{N}\{\boldsymbol{\mu}_b(\mathbf{y}_i; \phi), \boldsymbol{\Sigma}_b(\mathbf{y}_i; \phi)\}, \\ \{\boldsymbol{\mu}_b(\mathbf{y}_i; \phi), \boldsymbol{\Sigma}_b(\mathbf{y}_i; \phi)\} = \text{dnn}(\mathbf{y}_i; \phi, \boldsymbol{\lambda}_\phi) \end{array} \right.$$

▷ $\mathbf{y}_i$ are ‘covariates’ in this neural network $\Rightarrow$ *need to have the same length for all subjects*

4 DGMMs Estimation (cont'd)

Because of missing data y_i have unequal lengths

- We decompose y_i to
 - ▷ y_i^o observed measurements
 - ▷ y_i^m unobserved measurements

$$\{\mu_b(y_i^o, y_i^m; \phi), \Sigma_b(y_i^o, y_i^m; \phi)\} = \text{dnn}(y_i^o, y_i^m; \phi, \lambda_\phi)$$

We impute y_i^m under the *Missing At Random* assumption

4 DGMMs Estimation (cont'd)

Utilize *Data Augmentation* ideas

- Target posterior predictive distribution

$$p(\mathbf{y}_i^m \mid \mathbf{y}_i^o) = \int p(\mathbf{y}_i^m \mid \mathbf{y}_i^o; \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \mathbf{y}_i^o; \boldsymbol{\theta}) d\boldsymbol{\theta}$$

4 DGMMs Estimation (cont'd)

- We borrow ideas from the *Stochastic Approximation Expectation Maximization* algorithm

▷ Step 0: We set $\mathbf{y}_i^{m(0)}$ an initial value for $\mathbf{y}_i^m$

For iteration/epoch $it = 1, \dots, L$, we repeat the steps

▷ Step 1: Minimization

estimate $\boldsymbol{\theta}^{(it)}$ and $\boldsymbol{\phi}^{(it)}$ by minimizing $-\tilde{\ell}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{y}_i^{m(it-1)})$

▷ Step 2: Imputation

simulate $\mathbf{y}_i^{m(it)}$ from $[\mathbf{y}_i^m \mid \mathbf{y}_i^o; \boldsymbol{\theta}^{(it)}]$

4 DGMMs Estimation (cont'd)

- The predictive distribution $[y_i^m \mid y_i^o; \theta^{(it)}]$ does not have a closed-form, but is written as

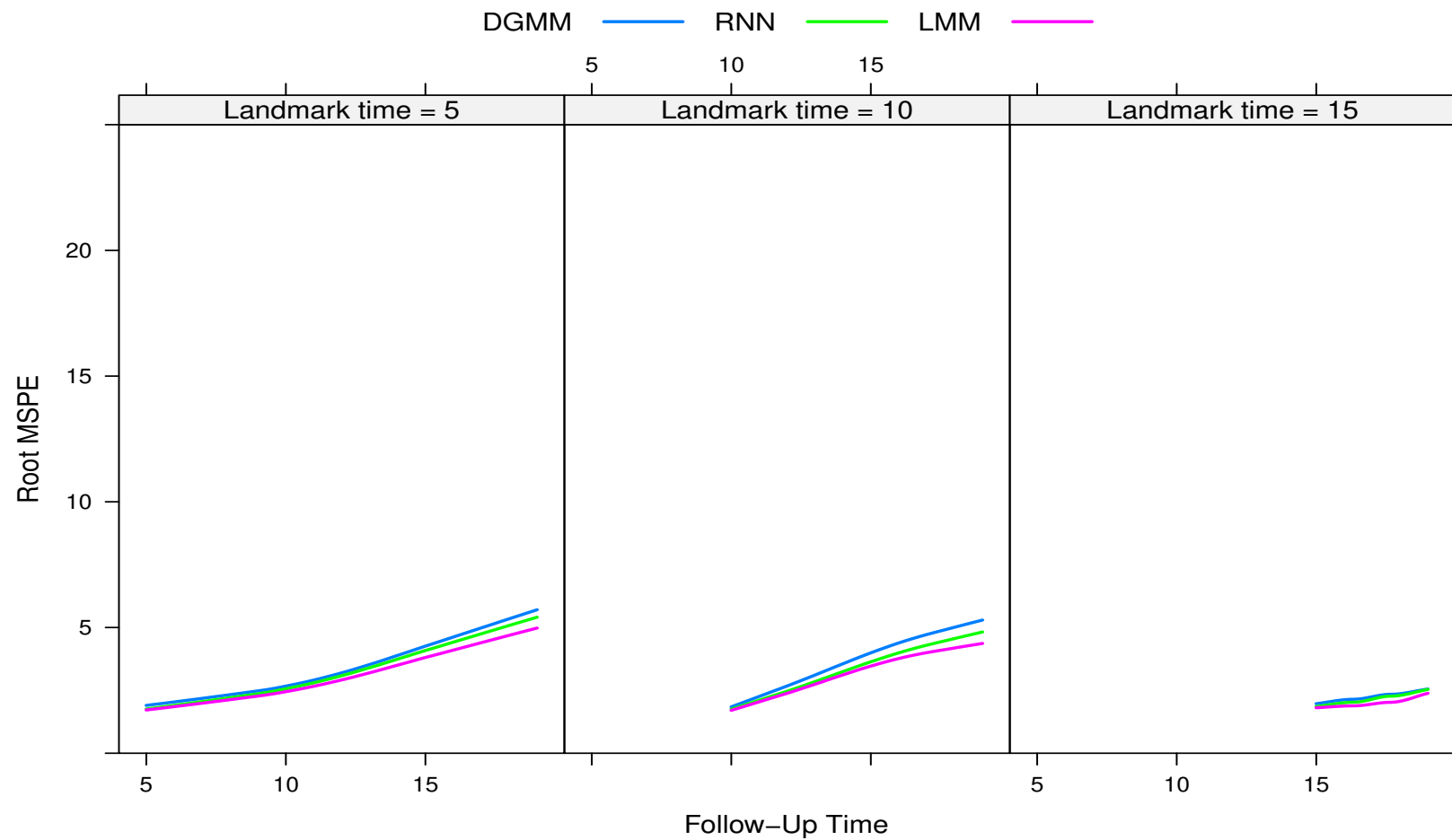
$$p(y_i^m \mid y_i^o; \theta^{(it)}) = \int p(y_i^m \mid b_i; \theta^{(it)}) p(b_i \mid y_i^o; \theta^{(it)}) db_i$$

- We use a Metropolis-Hastings algorithm
 - ▷ simulate b_i from $[b_i \mid y_i^o; \theta^{(it)}]$
 - ▷ simulate $y_i^{m(it)}$ from $[y_i^m \mid b_i; \theta^{(it)}]$

DGMM is implemented in Python using Tensorflow/Keras API

- Simulation Study I
 - ▷ one longitudinal outcome, nonlinear subject-specific profiles
 - ▷ $n = 800$, 20 measurements per subject
 - ▷ *no missing data*

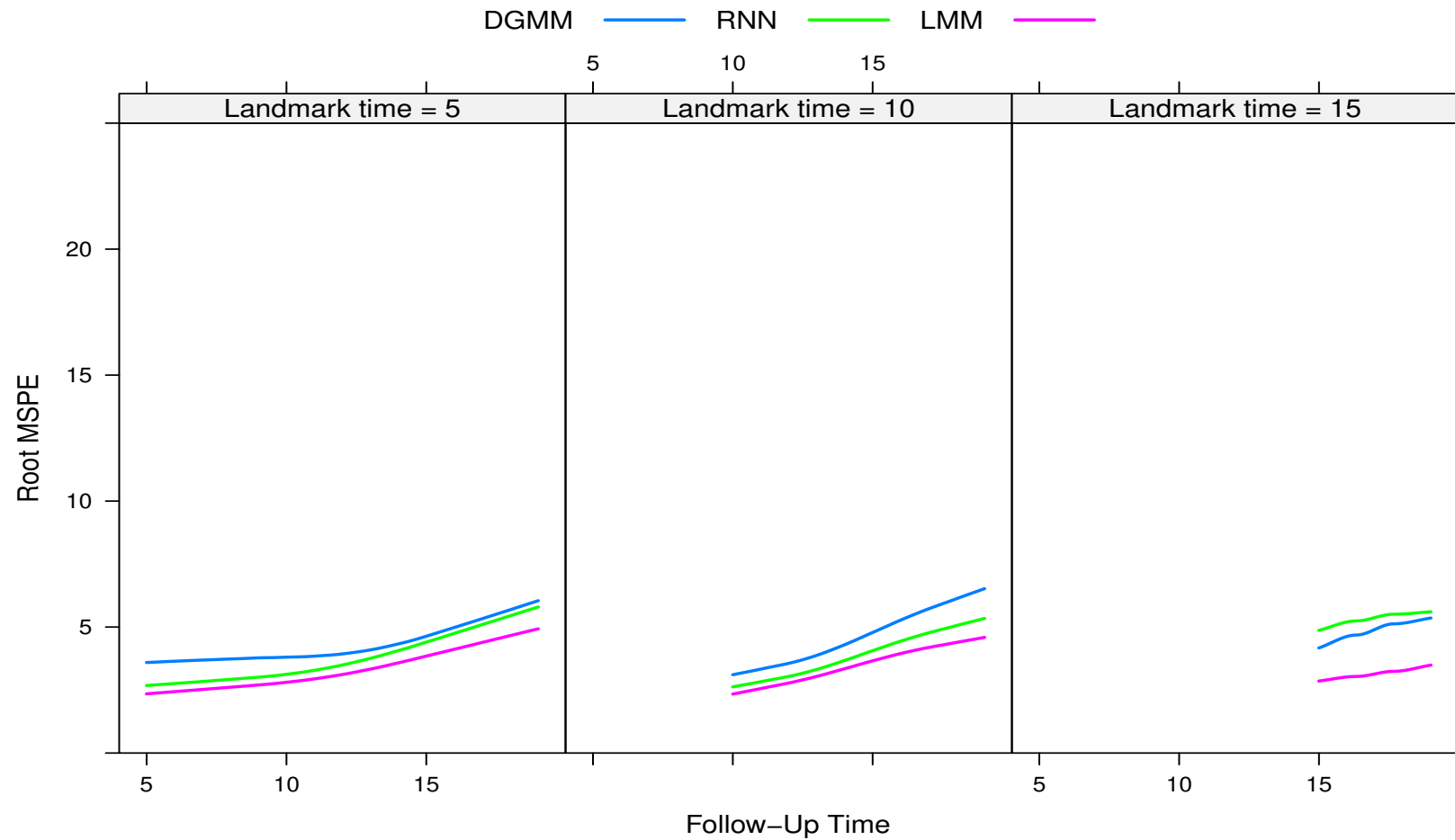
5 Simulations (cont'd)



5 Simulations (cont'd)

- Simulation Study II
 - ▷ one longitudinal outcome, nonlinear subject-specific profiles
 - ▷ $n = 800$, 20 measurements per subject
 - ▷ MAR missing data
 - ▷ 30% dropout

5 Simulations (cont'd)



6 Discussion & Future Plans

- DGGM shows good performance, but not yet as accurate as the linear mixed model
 - ▷ though it scales better for multivariate longitudinal data
- We are looking at ways to improve performance
 - ▷ replace dense with recurrent neural networks

6 Discussion & Future Plans (cont'd)

- Notes / lessons learned:
 - ▷ DNN's can be helped to model nonlinear profiles using functions of time (e.g., polynomials, cosine, etc.) in the specification of T_i
 - ▷ a full covariance matrix $\Sigma_b(\mathbf{y}_i; \phi)$ in the encoder $[\mathbf{b}_i \mid \mathbf{y}_i; \phi]$ works better than a diagonal matrix
 - ▷ *Posterior collapse*
 - * KL annealing
 - * $n_\phi > n_\theta$

6 Discussion & Future Plans (cont'd)

- Imputation step in DGMM not inherent to the fitting algorithm
 - ▷ *alternative*: pattern mixture imputation scheme
 - ▷ available case missing value restrictions correspond to MAR
 - ▷ decoder and encoder per dropout pattern
- Extend DGMM to fit joint models for longitudinal and event time data
 - ▷ event times are assumed to follow a generic distribution
 - ▷ baseline line hazard is approximated with a dnn
 - ▷ functional relationship between the hazard of an event and covariates and the linear predictor of the longitudinal model approximated with a dnn

Thank for your attention!

<https://www.drizopoulos.com/>